Progressive Retrieval Attention with FreeToken: Semantic Memory under Bandwidth-Adaptive MoE Serving

Jorge Simão

September 2026

Abstract

FreeToken adapts mixture-of-experts (MoE) inference to constrained machines by managing model/expert residency and movement. Progressive Retrieval Attention (PRA) manages a different sparse resource: semantic context selected for a query. We pin FreeToken commit `3a20a79038338c33bd051c52152e6d1faa4d9791`, audit its scheduler and cache architecture, implement a capability-honest adapter, and introduce a dependency-light transfer coordinator that keeps expert and semantic identities separate. A five-seed controlled sweep crosses four prefetch policies with 0.5–16 GiB/s host-device bandwidth. At 4 GiB/s, earliest-deadline coordination reduced mean exposed transfer latency to 22.3 ms, versus 27.1 ms for expert-only and 53.8 ms for independent fair sharing. At 16 GiB/s it hid both transfers in all seeds. At 1 GiB/s, however, PRA-only scheduling was better (162.8 versus 258.4 ms), falsifying any claim that simple coordination is universally best. The available RTX 5060 host lacks the CUDA 13 toolchain and sufficient host memory for the documented model set, so live E0/E2/E3 model qualification is precisely blocked. The reported result is a scheduler design and controlled frontier, not native FreeToken PRA execution. A separate M4 Pro MLX reference compares dense Qwen3-32B with Qwen3-30B-A3B: the MoE uses 26.4 rather than 70.3 MiB of all-layer selected K/V and completes substantially faster, but is more sensitive to removing early PRA consumer layers. Parameter sparsity does not therefore imply semantic-context sparsity.

1 Introduction

FreeToken and PRA are both selective, but they answer different questions. FreeToken asks which expert or model-state bytes must be resident to execute the model. PRA asks which evidence records and intervals should be visible to the query. Expert identity is a property of computation; semantic resource identity is a property of application state and authorization. Merging them because both use top-k decisions would make failures difficult to diagnose and could cross security boundaries.

The systems opportunity is physical coordination. Expert weights and selected PRA K/V may compete for the same PCIe link, host memory, device capacity, and prefetch window. A scheduler may improve utilization by considering both demand streams while preserving their separate policies.

This paper contributes a pinned architecture audit, separate SDK namespaces for semantic blocks and expert state, an optional native executor/E3 boundary, and a controlled bandwidth sweep that exposes both the promise and failure modes of simple coordination.

2 PRA and Integration Levels

E0 supplies selected evidence as visible text. E1 carries resource identity and lifecycle metadata. E2 consumes detached selected native K/V. E3 additionally requires the real FreeToken scheduler to own PRA placement, prefetch, sharing, and eviction. Only E3 permits claims about joint runtime scheduling.

Native PRA attention joins direct and memory keys before one softmax,

$$\mathrm{softmax}\Big([qK_D^\top; qK_M^\top]/\sqrt{d} + M\Big)\,[V_D; V_M]. \tag{1}$$

An expert cache hit cannot substitute for this contract: it makes model weights available, not evidence available.

Query → PRA router → semantic resource IDs and selected intervals → PRA text (E0) or native K/V (future E2).

Model token states → MoE router → expert IDs → CPU execution or expert transfer/cache.

The identities and selection losses remain independent. A physical coordinator receives only transfer size, deadline, residency, and predicted-use metadata, then arbitrates shared bandwidth. E3 begins only when FreeToken's real scheduler owns that arbitration and lifecycle.

Figure 1: Two selectors, one possible physical coordinator.

3 Pinned FreeToken Audit

At the pinned revision, FreeToken is an independent Python/CUDA serving stack with OpenAI and Anthropic APIs. Its CPU expert pool is a source of truth and a shared LRU cache holds selected expert state on the GPU. Prefill uses full-layer double buffering. During decode, bandwidth-adaptive logic divides expert misses between device transfer and CPU execution. The stack also contains paged/native KV pools, radix prefix reuse, recurrent-state checkpoints for applicable model families, and CUDA-graph-aware cache controls.

These mechanisms supply plausible extension seams but not native PRA by themselves. A semantic block needs a stable tenant/resource/version/model fingerprint, selected intervals, per-layer geometry, and request-scoped authorization. The expert cache uses different keys and retention semantics.

4 Implementation

`FreeTokenEngineAdapter` uses the server's OpenAI-compatible endpoint at E0. An explicit `FreeTokenNativeExecutor` may advertise E2 or E3; without it, the provider remains E0. Native semantic blocks live in `LogicalPRABlockStore`, never in an expert-ID namespace. Tests enforce this capability boundary.

The dependency-light coordinator represents each miss as

$$d_i = (c_i, b_i, t_i, h_i, u_i), \tag{2}$$

where $c_i \in \{\text{expert}, \text{PRA}\}$, b_i is bytes, t_i is first-use time, h_i is cache residency, and u_i records eventual use. We compare: expert-only prefetch, PRA-only prefetch, independent simultaneous prefetch with fair link sharing, and coordinated earliest-deadline transfer. Non-prefetched objects start at demand time and all policies share one physical link.

For completion time f_i, exposed transfer latency is

$$L = \sum_i \max(0, f_i - t_i), \tag{3}$$

and accelerator idle proxy is $\max_i \max(0, f_i - t_i)$. The model also reports expert/PRA bytes separately, ready-before-demand fraction, and wasted prefetch. It is deliberately algebraic: it can validate scheduling logic on a host where the full CUDA stack is unavailable, but cannot establish E3 throughput.

5 Experimental Method

Five deterministic seeds vary expert traffic from 96–192 MiB, PRA traffic from 12–48 MiB, expert first-use from 12–25 ms, and PRA first-use from 25–60 ms. Bandwidth is swept over 0.5, 1, 2, 4, 8, and 16 GiB/s. Each seed uses identical demands under every policy. The run was executed on Windows with an RTX 5060 Laptop GPU (8,151 MiB) and driver 592.19; the calculation itself is CPU-side.

The host has 12 GiB system memory, no CUDA 13 `nvcc`, and cannot satisfy the documented practical model configurations, which begin with models such as 20B/30B-class MoE systems. Installing a partial stack would not make a valid live-engine result. We therefore mark live E0, E2, and E3 as blocked on this host rather than using a mock model under the FreeToken name.

6 Controlled Results

The mean workload moves 152.9 MiB of expert state and 29.4 MiB of PRA state. At 4 GiB/s, coordinated scheduling improves over the best single-class policy by 17.7% and over independent fair sharing by 58.6%. At

Table 1: Mean exposed transfer latency (ms), five seeds. Bold is best at each bandwidth among the four fixed policies.

Bandwidth	Expert only	PRA only	Independent	Coordinated
1 GiB/s	258.4	**162.8**	296.1	258.4
4 GiB/s	27.1	37.3	53.8	**22.3**
16 GiB/s	1.79	9.33	0.68	**0.00**

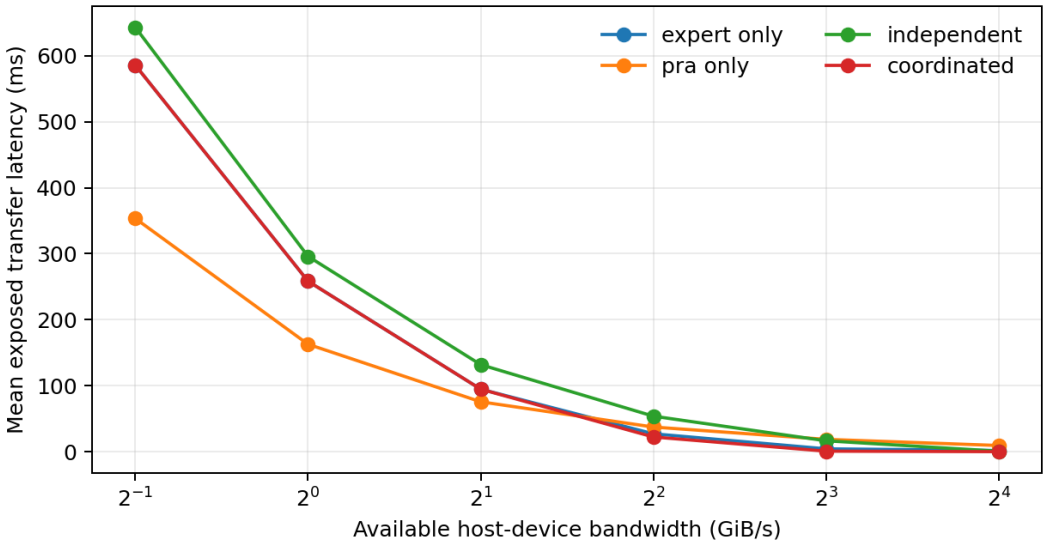

Figure 2: Controlled transfer frontier. Coordination becomes effective once the link can satisfy the early expert deadline and still stage PRA before use.

16 GiB/s all demands are ready before use under coordination. At 1 GiB/s the large early expert transfer monopolizes the link; prefetching the smaller later PRA object and executing the expert on demand minimizes the sum of lateness. An earliest-deadline policy is therefore not enough for low-bandwidth operation. The future scheduler needs a cost objective that considers bytes, slack, CPU fallback, and critical-path impact.

7 Dense-versus-MoE Architecture Reference

Before attributing any future result to FreeToken scheduling, we need to know how the underlying MoE changes the semantic-memory workload. We therefore reuse the selector-frozen MLX calibration owned by Papers 4.5 and 6.2. It runs five unique annotated-evidence examples from each of QASPER, HotpotQA, and 2Wiki for both Qwen3-32B and Qwen3-30B-A3B, with at most 384 selected source tokens. The comparison is model-backed and natural-QA-based, but it is *not* a FreeToken execution result.

All-layer concatenated transport reproduces the selected-text sequence for all 30 model–example pairs. The segmented one-softmax path agrees on 13/15 dense and 14/15 MoE examples; those mismatches are numerical execution sensitivity, because retrieval is frozen. More importantly for profile design, retaining only the last three quarters of consumer layers changes mean gold likelihood by .356 nats for dense 32B but 2.662 nats for the MoE. The MoE reduces weight-side computation and its native K/V topology is cheaper, yet the tested model needs early semantic-memory consumers more strongly. A FreeToken policy must keep expert residency and PRA consumer-layer selection as separate decisions.

8 Capability Matrix and Falsification

The central hypothesis remains falsifiable. Native PRA is not justified if an optimized selected-text path reaches the same quality-memory-bandwidth frontier, if semantic K/V traffic erases FreeToken's expert-offload benefit, or

Table 2: Dense and MoE reference on the 48 GB M4 Pro. PRA MiB is all-layer selected K/V. $|\Delta|$LP compares segmented native K/V with selected-text execution, averaged over dataset-level means.

Model	E0 F1	Model GiB	PRA MiB	E0 ms	Seg./E0	$\lvert\Delta\rvert$LP
Qwen3-32B	.231	17.16	70.3	1177.0	1.03	.039
Qwen3-30B-A3B	.256	16.00	26.4	222.5	1.12	.146

Table 3: Evidence available in this iteration.

Item	Status	Evidence
Architecture and extension seam	complete	pinned source audit
E0 adapter	implemented/tested	SDK unit tests
Bandwidth coordination	measured	controlled five-seed model
MLX dense/MoE reference	measured	30 natural-QA model–example pairs
Live FreeToken E0	blocked	host/toolchain/model gate
Native E2	not implemented	no claim
Scheduler E3	not implemented	no claim

if a joint scheduler cannot outperform independent policies under realistic deadlines. The 1 GiB/s result already rejects the naive stronger claim that coordination is always beneficial.

9 Limitations and Next Work

No *FreeToken* model generation, physical PCIe trace, expert hit trace, or native FreeToken attention path is measured here. The MLX reference isolates model architecture and cannot validate the scheduler. The synthetic demand distribution is a controlled instrument, not a workload proxy. The next iteration requires a CUDA-13-capable host with sufficient RAM for one documented model. It should first run FULL and frozen-selection E0, then add E2 with geometry and isolation controls, and finally inject the coordinator into the real scheduler. Fixed and oracle policies should precede any learned joint controller.

The live sweep must report task F1/EM, exact E0/E2 parity, selected tokens, expert and PRA bytes, link utilization, queue delay, ready-before-demand, wasted transfer, accelerator idle time, TTFT/ITL tails, throughput, and peak host/device memory. E3 is earned only when scheduler traces show ownership of PRA placement and cleanup.

10 Conclusion

FreeToken and PRA virtualize different resources. Their control policies should remain separate, but their transfers can sometimes be coordinated profitably. The controlled results locate a threshold: coordination helps at moderate and high bandwidth, while a simple deadline rule can lose under severe scarcity. The adapter and experiment make that hypothesis executable; the unavailable live engine configuration keeps E2/E3 claims open.

Artifact Availability

Branch `research/paper6-9-freetokens` contains the adapter, dependency-light coordinator, unit tests, pinned audit, raw five-seed JSON, plotting script, README, manuscript, and the vendored dense/MoE reference artifacts. The executable MLX runner is owned by `research/paper4-5-runtime` at commit `0f0ed60`; this branch does not duplicate that engine implementation.

References

[1] FlashML. *FreeToken: Affordable LLM Inference on Constrained Machines*. 2026. https://github.com/FlashML-org/FreeToken

[2] FreeToken authors. *FreeToken*. arXiv:2608.16157, 2026. https://arxiv.org/abs/2608.16157

[3] W. Fedus, B. Zoph, and N. Shazeer. *Switch Transformers*. JMLR, 2022.